Item 3: Relational Dynamics

3.1. Introduction to Dynamics in the Relational Model

3.2. Relational Algebra

- Basic RA: unary, compatible binary, combinations
- Extended RA: division, lossless join, sorting
- Analytic RA: grouping
- Complete RA: actualization

3.3. DML subset from SQL (dynamics)

Item 3.1: Relational Model Dynamics

- A *relation* is a **set** (of tuples). A DB is a collection of sets
- DB operators can either <u>inspect</u> or <u>change</u> (in Δt) the state of those sets

$$O(DB_i) = DB_{i+\Delta t}$$

* *the DB changes by dynamics*

- For doing those changes, it is required to interact through a language
- *Relational langs.* are domain specific, declarative and of specification type. Two subtypes can be distinguished:

  - *<u>Algebraic or procedural</u>*: changes are described *(one by one)* as data sets transformations (on DB elements or a copy of them) → ***relational algebra***

    Example: *find persons whose surname is 'Smith'*

  - *<u>Predicative or non-procedural</u>*: a goal is described, which can be attained (by valuating free variables) or not → ***relational calculus***

    Example: *Can persons(X, 'Smith') be satisfied? for what values of X?*

Item 3.2: Describing the path: Relational Algebra

- It is a Specification Language aimed at Manipulating Data Sets (conceptual description or ***design***, for later implementation)

- It was defined by E.F. Codd (1971) as a basis for the relational model. Initially, consisted of 5 *primitive* operators and 3 *derived* operators.

- Operators can also be classified into *unary*, *binary*, or *bin. compatible*. Operations take one or two relations and produce one relation.

- It is commonly used to describe sequences of transformations of sets (copies of the stored data in a workspace), aimed at satisfying a query.

- *All algebraic data languages are isomorphic*
  (you can use RA for designing queries, and then *translate* them into SQL).

Item 3.2: Relational Algebra – Unary Operators

Selection: choose tuples meeting some condition

Notation: $\sigma_{predicate}$ (Relation)

Books by Dumas

Books

Signat.	*Title*	*Author*
Dum-1	The Three Musketeers	Dumas
Per-1	Captain Alatriste	P-Reverte
Asi-1	The End of Eternity	Asimov
Dum-2	The Count of Monte-Cristo	Dumas

$\sigma_{author='Dumas'}$ (***Books***)

Signat.	*Title*	*Author*
Dum-1	The Three Musketeers	Dumas
Dum-2	The Count of Monte-Cristo	Dumas

(rows are chosen)

And, what about 'books either by Dumas or Pérez-Reverte'?

Item 3.2: Relational Algebra – Unary Operators

Projection: subset of the relation schema

Notation: $\pi_{atrib1,\ atrb2..}$ (Relation)

Books

Signat.	*Title*	*Author*
Dum-1	The Three Musketeers	Dumas
Per-1	Captain Alatriste	P-Reverte
Asi-1	The End of Eternity	Asimov
Dum-2	The Count of Monte-Cristo	Dumas

→

$\pi_{title,author}$ (***Books***)

Title	*Author*
The Three Musketeers	Dumas
Captain Alatriste	P-Reverte
The End of Eternity	Asimov
The Count of Monte-Cristo	Dumas

(columns are chosen)

What about projecting by author?
What should be the relation's degree? And its cardinality?

Item 3.2: Alias definition – *Renaming* (ρ)

<u>Rename</u>: assign the result of an expression to any symbol (temporal relation which existence is restricted to the query being defined)*.

Notation: ρ_{symbol} (Expression), or also written as $S \equiv$ Expression

Let be Q the set of titles of books written by Dumas

$$\rho_A \ (\sigma_{author='Dumas'} (\textit{Books}))$$

$$A \equiv \sigma_{author='Dumas'} (\textit{Books})$$

$$\rho_Q \ (\pi_{title} \ A)$$

$$Q \equiv \pi_{title} \ A$$

*: for temporal tables with longer persistence (session), the assign operator ← is used

Item 3.2: Relational Algebra – Set Operators

Unary operators take one operand; Binary operators take two; **Binary Compatible** operators take two with <u>common definition</u>:

- *same degree*
- *same data nature (two by two)*

- If the relations to be operated aren't compatible, you can apply projections:

ADVENTURE (<u>Signat</u>, Title, Author, Type)

not compatible!

NOVELS (<u>Signat</u>, Title, Author)

A: $\pi_{\text{Signat, Title, Author}}$ (ADVENTURE) *compatible!* NOVELS

B: ADVENTURE *compatible!* $\pi_{\text{Signat, Title, Author, "Novel" as type}}$ (NOVELS)

- *result's labels are assumed to be those of the first operand*

Item 3.2: Relational Algebra – Set Operators

Union: all the tuples of both (compatible) relations , sparing repeated tuples.

Adventures

Signat.	*Title*	*Author*
Dum-1	The Three Musketeers	Dumas
Per-1	Captain Alatriste	P-Reverte
Dum-2	The Count of Monte-Cristo	Dumas

Novels

Signat.	*Title*	*Author*
Ner-1	Twenty Love Poems	Neruda
Asi-1	The End of Eternity	Asimov

→

Adventures $\cup$ ***Novels***

Signat.	*Title*	*Author*
Dum-1	The Three Musketeers	Dumas
Per-1	Captain Alatriste	P-Reverte
Asi-1	The End of Eternity	Asimov
Dum-2	The Count of Monte-Cristo	Dumas
Ner-1	Twenty Love Poems	Neruda

Adventures and novels

Item 3.2: Relational Algebra – Set Operators

<u>Intersection</u>: all the tuples within **both** (<u>compatible</u>) relations

Adventures

Signat.	*Title*	*Author*
Dum-1	The Three Musketeers	Dumas
Per-1	Captain Alatriste	P-Reverte
Dum-2	The Count of Monte-Cristo	Dumas

Novels

Signat.	*Title*	*Author*
Ner-1	Twenty Love Poems	Neruda
Per-1	Captain Alatriste	P-Reverte

→

Adventures $\cap$ ***Novels***

Signat.	*Title*	*Author*
Per-1	Captain Alatriste	P-Reverte

Adventure Novels

Item 3.2: Relational Algebra – Set Operators

<u>Difference</u>: tuples that appear in the first relation, but are not present in the second one.

Adventures

Signat.	*Title*	*Author*
Dum-1	The Three Musketeers	Dumas
Per-1	Captain Alatriste	P-Reverte
Dum-2	The Count of Monte-Cristo	Dumas

Novels

Signat.	*Title*	*Author*
Ner-1	Twenty Love Poems	Neruda
Per-1	Captain Alatriste	P-Reverte

→

Adventures - Novels

Signat.	*Title*	*Author*
Dum-1	The Three Musketeers	Dumas
Dum-2	The Count of Monte-Cristo	Dumas

Adventure books which are not classified as novels

Item 3.2: Relational Algebra – Join Operators

Cartesian Product: every combination of tuples of both relations

Dishes

Specialty	*Region*
Gazpacho	Andalucía
Cocido	Castilla
Pote	Galicia

Wines

Denomination
Rioja
Rueda

→

Dishes* x *Wines

Specialty	*Region*	*Denomination*
Gazpacho	Andalucía	Rioja
Gazpacho	Andalucía	Rueda
Cocido	Castilla	Rioja
Cocido	Castilla	Rueda
Pote	Galicia	Rioja
Pote	Galicia	Rueda

Feasible Menu with every wine that is offered

Item 3.2: Relational Algebra – Join Operators

<u>Join</u> (*simple join* or *inner join*): subset of the Cartesian product meeting some conditional expression (that includes comparators =, >, < on either attributes and/or values; and logical connectors).

Which movies on theaters can watch our clients?

Clients

Name	*Age*
Fulano	29
Mengano	18
Zutano	9

Films

Title	*Rated*
Matrix	18
Only you	0

→

Clients $\theta_{\text{Age} \geq \text{Qualification}}$ ***Films***

Name	*Age*	*Title*	*Rated*
Fulano	29	Matrix	18
Fulano	29	Only you	0
Mengano	18	Matrix	18
Mengano	18	Only you	0
Zutano	9	Only you	0

Item 3.2: Relational Algebra – Join Operators

<u>Natural Join</u>: particular case of join when the applied operator is the equality (very useful for joining relations associated by foreign keys).

Employees' name who own a car

Employees

Name	*DNI*
Fulano	569064
Mengano	434848
Zutano	383090

Cars

Mark	*owner*
Ford Escort	383090
Seat Ibiza	569064

→ $\textit{Employees} \underset{\textbf{owner}}{*} \textit{Cars}$

Name	*DNI*	*Mark*
Fulano	569064	Seat Ibiza
Zutano	383090	Ford Escort

→ $\pi_{\text{name}}(\textit{Employees} \underset{\textbf{owner}}{*} \textit{Cars})$

Name
Fulano
Zutano

• *Notice that the repeated column is removed*

Notations: *RelationA* * *RelationB*, or also *RelationA* ⋈ *RelationB*

- <u>without subindex</u>: (a) equijoin by any homonymous attribute; (b) equijoin by foreign key (use only when there is no confusion).
- <u>with a key</u>: (a) equijoin by the specified homonymous attributes; (b) equijoin by the specified foreign key (use only when there is no confusion).
- **with conditional expression: equijoin by the specified condition**

Item 3.2: VALID way ≠ PROPER way

ρ

Renaming is also known as *using temporal views*, that is, temporal **constant** workspaces that support the main query. Thus, by means of renaming, you can apply that common computing motto coined by Caesar: *'Divide and Conquer'*. Nested queries are hard, and can be often avoided by renaming.

θ

*

Joining can be derived into two operators (σ×), but by joining you are doing both at the same time. Imagine you have 100 readers and 20 age graded books, and you need to list proper combinations (200). Will you write down 2000 combinations and then scratch out 1800 wrong lines? Or write down only the tallies?

Item 3.2: Relational Algebra

Primary and Derived Operators

- Some relational algebra operators can be substituted by a sequence of some other algebraic operations. The first are named *derived operators*.
- On the contrary, primitive operators are those that cannot be obtained by combining other primitive operators.

Primitive	**Derived**	
• Selection	• Intersection	A - (A-B)
• Projection	• Join	σ… (AxB)
• Cartesian Product	• Natural Join	π... σ… (AxB)
• Difference		
• Union		

Item 3.2.2: Extended Rel. Algebra: Division

Division: the quotient comprises any tuple meeting that *its cross join with the divisor is a subset of the dividend.*

Notation:

$$\mathbf{A \div B \equiv \pi_{sch(A)-sch(B)} A - \pi_{sch(A)-sch(B)} ((\pi_{sch(A)-sch(B)} A \times B) - A)}$$

Curriculum

Candidate	*Ability*
Pepe Pérez	Language C
Pepe Pérez	Java
José Gómez	Language C
José Gómez	SQL
Fulano Pi	SQL
Fulano Pi	Java
Fulano Pi	Language C

Position

Profile	*Requisite*
Analist/Dev	Language C
Analist/Dev	SQL

Candidates meeting the profile 'Analist/Dev'

$$\text{Curriculum} \div \pi_{requisite} (\sigma_{profile='Analist/Dev'} \text{Position})$$

→

Candidate
José Gómez
Fulano Pi

Item 3.2.2: Extended Rel. Algebra: SEMI-JOIN

<u>Semi-join</u>: same as any join, but only columns from one operand are observed. It has to state either left (|*) or right (*|) operand.

Notation: left semijoin: |* or $\ltimes$ right semijoin: *| or $\rtimes$

Employees

Name	*DNI*
Fulano	569064
Mengano	434848
Zutano	383090

→

Employees $\ltimes_{\text{owner}}$ ***Cars***

Name	*DNI*
Fulano	569064
Zutano	383090

Employees with a car

Cars

Make/Md	*Owner*
Ford Escort	383090
Seat Ibiza	569064
Honda Civic	868015

→

Employees $\rtimes_{\text{owner}}$ ***Cars***

Make/Md	*Owner*
Ford Escort	383090
Seat Ibiza	569064

Cars of our employees

Subindex descriptions: same meaning as in the equijoin.

Equivalence: $\ltimes \equiv \pi \bowtie$

Item 3.2.2: Extended Rel. Algebra: ANTI-JOIN

<u>Anti-Join</u>: same as semi-join, but the tuples kept are those not meeting the specified condition.

Notation: left antijoin: $\triangleright$ right antijoin: $\triangleleft$

Employees

Name	*DNI*
Fulano	569064
Mengano	434848
Zutano	383090

→

Employees $\triangleright_{\text{owner}}$ ***Cars***

Name	*DNI*
Mengano	434848

Employees without a car

Cars

Make/Md	*Owner*
Ford Escort	383090
Seat Ibiza	569064
Honda Civic	868015

→

Employees $\triangleleft_{\text{owner}}$ ***Cars***

Make/Md	*Owner*
Honda Civic	868015

Parked cars which are not own by any of our employees.

Subindex descriptions: same meaning as in the equijoin.

Equivalence: $A \triangleright B \equiv A - (A \ltimes B)$

Item 3.2.2: Extended Rel. Algebra: Outer Join

Outer Join: extension of join operator for observing non related tuples (those belonging to the antijoin). The result is the equijoin plus the left/right/both antijoin (padded with NULL or ω values).

left outer join:]* or ⟕ **right outer join**: *[or ⟖ **full outer join**:]*[or ⟗

Employees

Name	*DNI*
Fulano	569064
Mengano	434848
Zutano	383090

Cars

Make/Md	*Owner*
Ford Escort	383090
Seat Ibiza	569064
Honda Civic	868015

Employees with their cars, in case

Employees $\rtimes_{owner}$ ***Cars*** →

Name	*DNI*	*Make/Md*
Fulano	569064	Seat Ibiza
Zutano	383090	Ford Escort
Mengano	434848	NULL

Cars and owner (if s/he is an employee)

Employees $\ltimes_{owner}$ ***Cars*** →

Make/Md	*Owner*	*Name*
Ford Escort	383090	Zutano
Seat Ibiza	569064	Fulano
Honda Civic	868015	NULL

All employees and cars, linked to the extend of our knowledge

Employees $⟗_{owner}$ ***Cars*** →

Name	*DNI*	*Make/Md*
Fulano	569064	Seat Ibiza
Zutano	383090	Ford Escort
Mengano	434848	NULL
NULL	868015	Honda Civic

Item 3.2.2: Extended Rel. Algebra: Sorted Sets

Sorted Set (list) is the result of applying either an ascending ($\top_{criteria}$) or descending ($\overline{\top}_{criteria}$) sorting criteria on a relation. For *multiattribute criteria* on opposite directions, several operators are used ($\overline{\top}_{age}\ \top_{surname}$)

Whenever a sorted list is operated, the result is a not sorted relation.

Some aggregation (analitic) functions can be applied on sorted lists: *first*, *last*, and *rank(value)*, and the pseudo-column *numrow.*

Persons

Name	*Age*	*Nationality*
Fulano	29	Spanish
Zutano	3	Spanish
John Doe	73	English
Smith	14	English
Mengano	49	Spanish
Pelancejo	25	Spanish

$\sigma_{first(3)}\ \overline{\top}_{age}$ ***(Persons)***

Name	*Age*	*Nationality*
John Doe	73	English
Mengano	49	Spanish
Fulano	29	Spanish

Item 3.2.3: Selective vs Analytic Queries

How old are you? Yes, you.

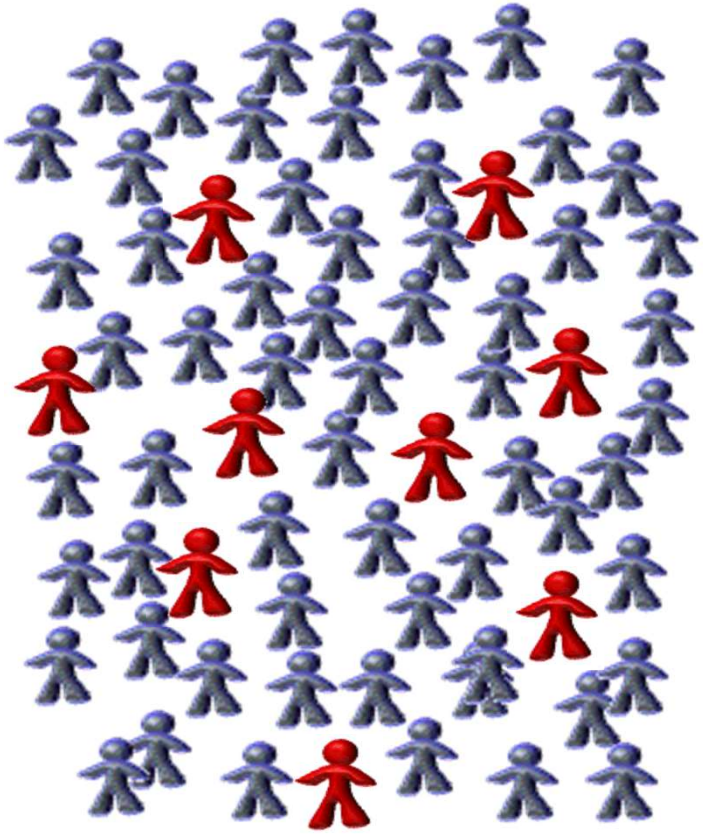
$\pi_{age}\ \sigma_{id='you'}$ (PEOPLE)

How old are you? All of you.

$\pi_{AVG(age)}$ (PEOPLE)

AGGREGATE

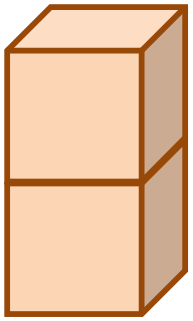
Item 3.2.3: Grouped Analytic Queries



How old are all of you?

Er, wait! How many are you?

Group by color...



COUNT	AVG(age)
9	47.33
61	27.54

$$\pi_{\text{COUNT, AVG(age)}}\ \mathcal{G}_{\text{color}}\ (\text{PEOPLE})$$

$$\pi_{\text{AVG(age)}}\ (\text{PEOPLE}) \equiv \pi_{\text{AVG(age)}}\ \mathcal{G}_{\text{null}}\ (\text{PEOPLE})$$

*Note: after grouping, original data is **no longer** in the workspace…*

Item 3.2.3: Rel. Algebra – Grouping Operator

Grouping:
groups are built following a given criteria, which is a subset of the relation schema, for applying some aggregation function.

Notation: $\pi_{\texttt{projection}}\ \sigma_{\texttt{selection}}\ \mathcal{G}_{\texttt{criterion}}\texttt{(relation)}$

Example: *How many persons are there for each nationality?*

Person

Name	*Age*	*Nationality*
Fulano	29	Spanish
Mengano	49	Spanish
John Doe	73	English
Smith	14	English
Zutano	3	Spanish
Pelancejo	25	Spanish

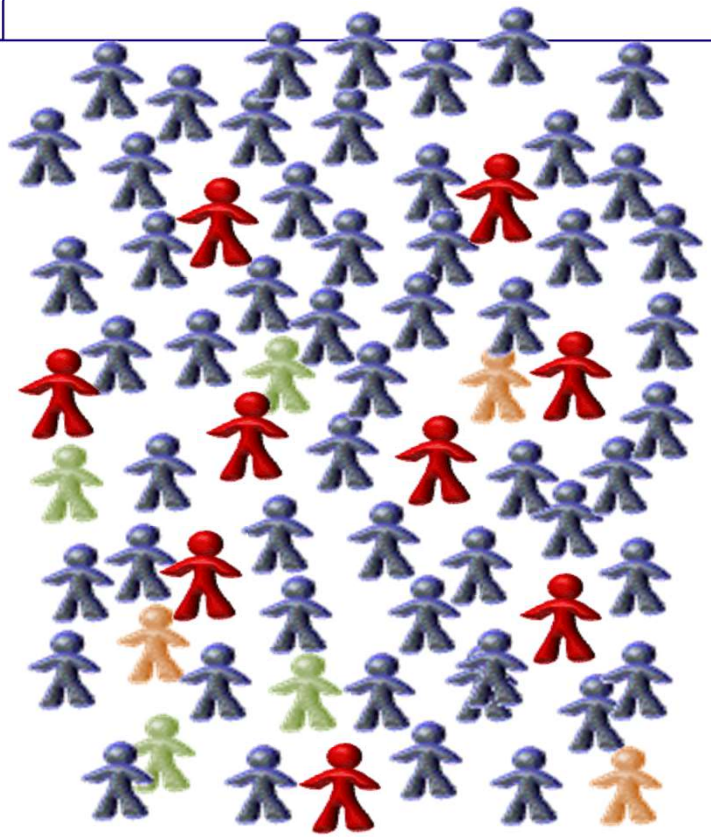
$$\pi_{\text{count ('x')}}\ (\mathcal{G}_{\text{Nationality}}\ \textbf{(Person)})$$

→

Nationality	*Count*
Spanish	4
English	2

Aggregation Functions: count(), sum(), avg(), min(), max(), …

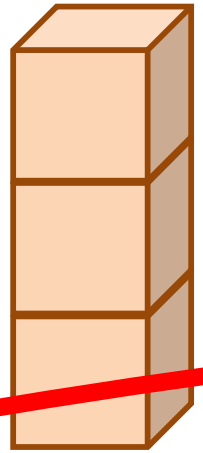
Item 3.2.3: Individual & Collective Selections



How old are you who are not orange?

Group by color...

Abstain groups with less than five



COUNT	AVG(age)
9	47.33
54	26.85
~~4~~	~~32.25~~

$$\pi_{\text{COUNT, AVG(age)}}\ \sigma_{\text{COUNT} >= 5}\ \mathcal{G}_{\text{color}}\ (\sigma_{\text{color} \neq \text{'orange'}}\ \text{PEOPLE})$$

collective ≠ individual

Item 3.2.4: Rel. Algebra for Actualizing Assignment and Tuple Update

Assignment: it is used for defining update operations on the DB. The operator notation is ←

Examples (relations update):

Persons ← Persons – ($\sigma_{name='José'}$ (*Persons*))

Persons ← ∅

Persons ← Persons ∪ {('Fulano', 'Pérez', 'Mengánez')}

Persons ← Persons ∪ ($\pi_{name,\ surname}$ (*Clients*))

Persons ← $\pi_{'Juan',surname}\ \sigma_{name='John'}$(*Persons*) ∪ $\sigma_{name!='John'}$(*Persons*)

Alternative Notation (**update**): $\delta_{name\leftarrow'Juan'}(\sigma_{name='John'}(Persons))$

Temporal table creation: temp ← Persons

Item 3.3: SQL for Dynamics - DML

SQL3 sentences for manipulation can operate in three ways:

1. Interactive: directly indicating SQL sentences
2. Embedded SQL: sentences are embedded in another host language, such as C or JAVA, supported by libraries or a *connection object*.
3. By modules (explicit procedure calls from an external process).

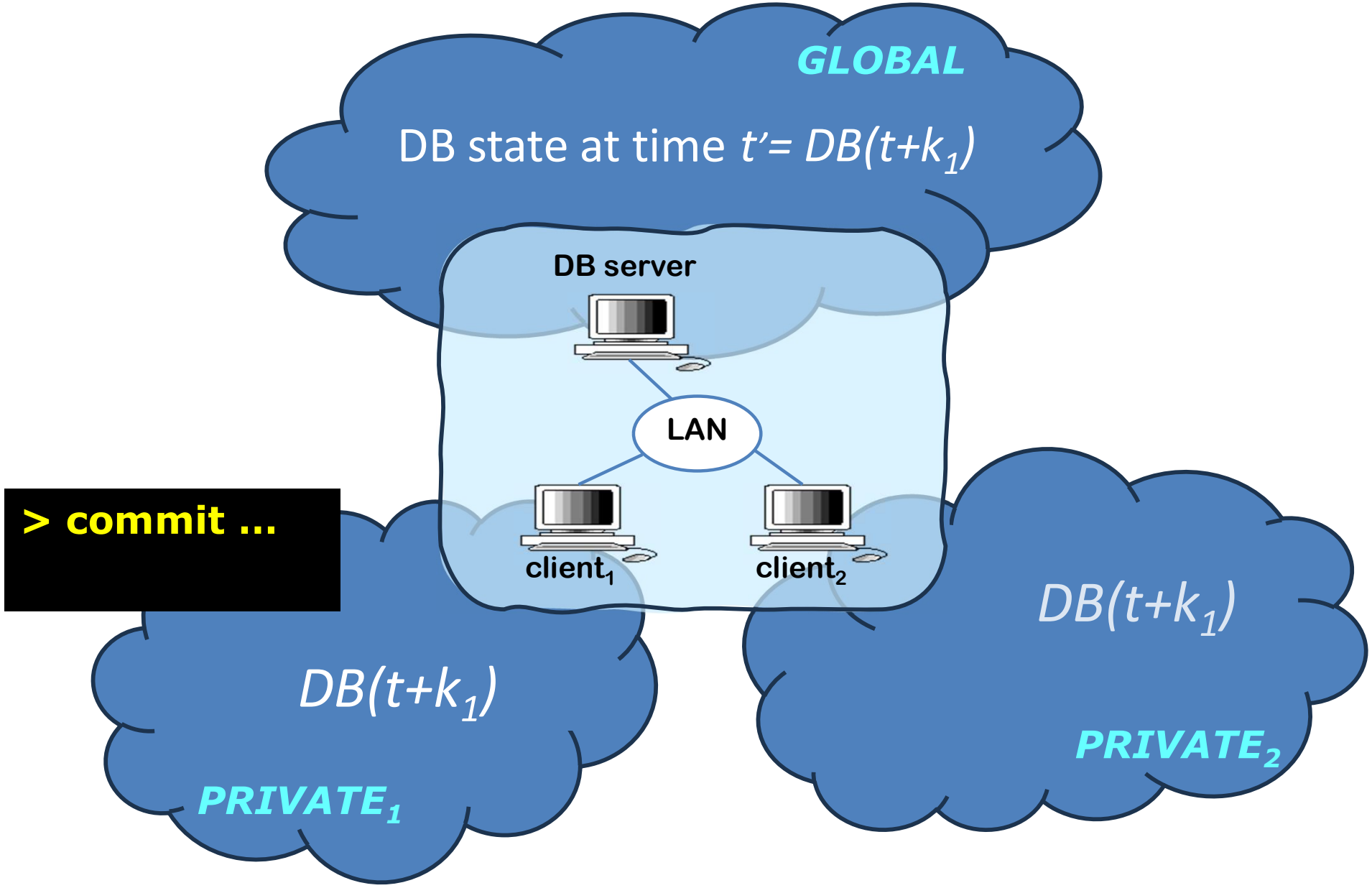
Actualization Operations

- Tuples insertion (INSERT)
- Tuples deletion (DELETE)
- Tuples modification (UPDATE)

Retrieval Operations

- Queries (SELECT)

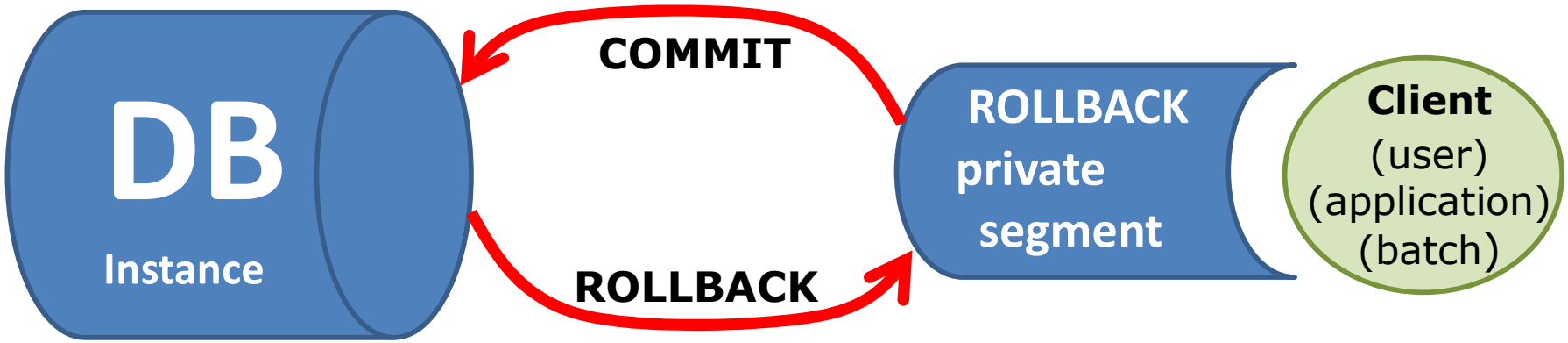
Item 3.3: Transactional Behavior



Item 3.3: Data Control Language: DCL

Transactional Management with PL/SQL

- **Transaction**: set of **actualization** instructions which must be run atomically (as a whole, "all or nothing")
- Instructions: COMMIT (perform) and ROLLBACK (undo)
  - `COMMIT [WORK]`
  - `ROLLBACK [WORK] [TO [SAVEPOINT] <savepoint>]`
  - `SAVEPOINT <savepoint>`



Item 3.3: DML for QUERYing: the SELECT

```
[WITH
      <symbol> AS <subquery>
      [, <symbol> AS <subquery> … ] ]

SELECT [ALL|DISTINCT] <selection_list>

      FROM <source_clause>

      [WHERE <condition> ]

      [GROUP BY <criterion> [HAVING <condition>]]

      [{UNION|UNION ALL|MINUS|INTERSECT} <query>]

      [ORDER BY <criterion> [ASC|DESC]] ;
```

Item 3.3: Transition Rel. Algebra ↔ SQL

```
[WITH
      <symbol> AS <subquery>
      [, <symbol> AS <subquery> … ] ]

SELECT [ALL|DISTINCT] <selection_list>

      FROM <source clause>

      [WHERE <condition> ]

      [GROUP BY <criterion> [HAVING <condition>]]

      [{UNION|UNION ALL|MINUS|INTERSECT} <query>]

      [ORDER BY <criterion> [ASC|DESC]] ;
```

ρ

π

× ⋈ ⟕ ⟖ ⟗ θ ∪

⋉ ▷

σ

right

left

Ǵ

⊤ ⊤

∪ ∩ −

Item 3.3: Running Order

```
[WITH
  1  <symbol> AS <subquery>
     [, <symbol> AS <subquery> … ] ]

SELECT [ALL|DISTINCT] <selection_list>      7

  2  FROM <source_clause>

  3  [WHERE <condition> ]

  4  [GROUP BY <criterion> [HAVING <condition>]]

  5  [{UNION|UNION ALL|MINUS|INTERSECT} <query>]

  6  [ORDER BY <criterion> [ASC|DESC]];
```

Item 3.3: `<selection list>` is SQL Projection

"Quantifiable list of workspace elements, separated by commas", such as:

- the **wildcard** (*) which means *all* (can be table.*, applied on a table)
- **attributes** of the workspace's relation scheme
- pseudo-columns, such as ROWNUM or table.ROWID,...
- constant values (as 1 or 'x') and (bound) variables
- calls to **functions**:
  - arithmetic (+, -, *, /), string (||, substr, onstr,...),
  - codification (nvl, decode, case, ...)
  - conversion: to_char, to_date, to_number
  - system functions, such as nullary SYSDATE and USER
  - aggregation functions (analytic queries)
  - user defined functions

- Any element (column/table) can be renamed (alias): ... *element* [AS] *alias* ...

Item 3.3: the WORKSPACE

It's a temporal table (linked to data physical location)

- It's defined in the FROM clause and can comprise a single table or several of them properly combined (which is actually just a table).
- Since the workspace is a table, a particular case of table in this clause is another workspace, that is, a complete query (or rather, a *subquery*).
- It allows table renaming (obliged for *self-joins*): **…** ***table alias*** **…**
- There exist several types of table combination (use most appropriate).

- **Basic Combination: the Cartesian Product** (avoid if possible)

```
… FROM people CROSS JOIN clients… ≡ FROM people, clients…
```

Item 3.3: Implementing a WORKSPACE

- **General JOIN syntax:** provides all columns in both tables

```
... FROM people [INNER] JOIN clients
            [ ON (<col_a>=<col_b> [AND...] )]
```

- without specification: equijoin by all columns named identically in both tables
- with ON clause: only rows matching expression. Can be =, !=, <, >, …

- **Natural JOIN:** does not provide twice the columns in the joining key

```
... FROM X [NATURAL [INNER]] JOIN Y...
```

Natural Join by every column named identically in both tables

```
... FROM X [INNER] JOIN Y USING (<columns>)...
```

Natural Join by the specified columns, named identically in both tables

Item 3.3: Implementing a WORKSPACE

- **Lossless combination: the OUTER JOIN**

```
...FROM people {LEFT|RIGHT|FULL} [OUTER] JOIN Clients
     [USING <columns> | ON <col_a>=<col_b> [AND...]]
```

- Same use of conditional clauses as with Natural Joins

- `... FROM X LEFT OUTER JOIN Y...`

Left Outer Join: respecting all tuples in the first table; missing columns will have null value

- `... FROM X RIGHT OUTER JOIN Y...`

Right Outer Join: respecting all tuples in the second table

- `... FROM X FULL OUTER JOIN Y...`

(Full) Outer Join: respecting all tuples in both tables

Item 3.3: SQL Language – SELECTION

➢ WHERE: conditional expression can be…

Type of Conditional Expression	Conditional operators
comparison	= != < > <= >= operands can be values, list of expressions, or even a subquery
null value test	<expression> IS [NOT] NULL
alike test (*reg. expression*)	<char_exp> [NOT] LIKE <pattern>
logical expression	{NOT, AND, OR} from other conditional expressions
inclusion test (in a range)	<expressn> [NOT] BETWEEN <exp> AND <exp>
inclusion test (set or subquery)	<expression> [NOT] IN {<expression_list> | subquery }
existence test	EXISTS subquery

SUBQUERIES in the WHERE clause: could be inefficient

- EXISTS stops at first match, but runs **nested** always.
- IN could be optimized (not run for every row); better moved to WITH clause
- NOT IN could be used for anti-joins (when efficient)

Item 3.3: Grouping & Sorting: Examples

group	name	surname	score	age
2º	Fulano	Pérez	113	32
1º	Mengana	Gómez	47	19
1º	Zutano	Smith	89	44
2º	Peranceja	García	45	10
1º	John	Doe	67	78

```
SELECT group, count('x'),
       avg(age), sum(score)
       FROM people
       GROUP BY group;
```

group	count	AVG(age)	SUM(score)
2º	2	21	158
1º	3	47	203

```
SELECT * FROM people ORDER BY apellido;
```

group	name	surname	DNI	age
1º	John	Doe	6789	78
2º	Peranceja	García	2345	10
1º	Mengana	Gómez	4567	19
2º	Fulano	Pérez	0123	32
1º	Zutano	Smith	8901	45

NEW workspace !

Item 3.4: Validation of a Selection

- Database operations allow you to know the current state or modify it (after a time interval Δt): $O(DBt) = DB_{(t + \Delta t)}$
- Their results can be **verified**. In addition, some operations (those involving selection) are subject to interpretation and must be **validated**.
- Selective operations require formalization (algebra/SQL) of a **conceptualized need** (from any agent: consumer; analyst/designer; data scientist) expressed in natural language, so that data engineers can implement it (directly or with tools).
- Deviations can occur in these processes of conceptual communication and implementation, which must be limited/fixed through validation tests.
- Unvalidated (or insufficiently validated) selections are a common source of errors and can lead to disastrous consequences.
- Remember: ***syntactic correctness does not imply validity***

Item 3.4: Data Testing (I)

- It's the basis for validating the design of a *query* in a given case.
- To validate a *query*, it is necessary **to identify all relevant cases** and develop (at least) one ***data test*** for each of them.
- Some of the most common practices include the following points:
  - Identify the affected relations and their **keys** (selection, projection, joining, grouping, aggregation, ordering, etc.).
  - For each key, analyze its domain (boundaries, regions, inflection points, and outliers) and then define the evaluation cases.
  - It is often helpful to draw a table (*subjected keys* in columns, *test cases* in rows). It is advisable to add one test (row) for each evaluation case analyzed and then simplify (if several tests fit within a single case).

Item 3.4: Data Testing (II)

- Other common practices are included in the following points: (cont.)
  - Discern when an attribute takes different values in two separate tests, and when it takes different values in different tuples coexisting in the same test.
  - In relations with multi-attribute PK, several tuples will be required for each attribute of the primary key (fixing the value of this attribute and then searching for all relevant combinations of values for the other attributes). Example: in PURCHASES(id, date, ref), list *customers who have not bought soda*. It becomes necessary to have several tuples for the same *id* with different *ref* (and with the same ref; and for the same and different dates; etc.).
  - Also, when the PK of cursors (result/temp.views) is not the relation's PK. Example: in PURCHASES(id, ref, city), cities where soda has not been purchased.
  - Check validity of algebraic operations (looking for diversity in the cases):
    - removal of duplicates
    - compatible binary operations
    - grouping
    - sorting

Item 3.4: Data Test Definition

- Any *data test* is based on running a selection operation on a known DB state (for which the operation's outcome is known).
- If the *obtained result* matches the *expected result*, the test **fails**. Conversely, if the test **succeeds**, divergences must be documented (so that analysts/developers can study its implications, locate bugs, and correct defects).
- Some data tests may start from a trivial database state (an empty database).
- Typically, a controlled (known) *data load* is performed from this trivial state, for which the query result is known.
- Another typical definition of a data test involves starting from a database state (for which the query is executed, obtaining a result $R(BD_t)$) and then defining a sequence of actualizing operations whose effect on the query result is known (that is, the result $R(BD_{(t+\Delta t)})$ can be predicted).

  Example: in a given DB state, the query `select count(x) from clients;` returns 45; if another row is inserted into *clients*, the new expected resulted will be 46.